

INTRODUCTION

Background of the Project

Traffic management is among the important aspects of modern transport systems, especially for urban areas where the number of vehicles keeps increasing with each passing day. Traffic signals are among the most significant tools used in controlling traffic flows. They ensure that traffic moves smoothly from one point to another without collisions and congestions. However, some conventional traffic signaling systems have been developed based on fixed timings which often lead to unnecessary delays and congestions.

Most traffic signaling systems utilize discrete integrated circuit technologies. They make use of different electronic components such as timers, counters, and logic gates in order to work. Even though the traffic signaling systems using such technologies have proven to be very efficient, they are often very difficult to develop. There is therefore need to look for alternatives that may offer the same level of efficiency but are easier to build and modify if necessary.

This paper focuses on the design and development of a timed traffic signaling system using the Arduino microcontroller technology. Unlike other conventional traffic signal systems, this signaling system does not involve any external electronic parts. All controls will be handled by the software only. Thus, the system will have no IC parts except for those used in Arduino. The system runs based on different state timings, thus making it a sequential logic system based on Arduino programming.

Relevance of Digital Logic in Embedded Systems

The field of digital logic is very important to embedded systems because it defines how a certain system accepts inputs and produces corresponding outputs. The Boolean expression, truth table, and sequential logic are some terms that define how the embedded system behaves with different stimuli.

For example, in a traffic light embedded system, the operation of the light from red, yellow, and green lights and their timings follow digital logic. Using FSM, a state machine is created whereby the traffic light transitions from one light to another based on defined timings.

This project makes use of the above principles by developing them via software programming through an Arduino platform. Instead of creating physical components for logic like ICs and other logic gates, these are made via programming.

Connection Between Logic Design and Arduino Implementation

The application of digital logic design becomes very easy and feasible using Arduino. While traditional digital logic design relies on electronic components, such as gates, the same processes are executed by code in an Arduino. Digital logic design is used in designing this project. This means that the operation of the traffic light system is modeled using Boolean expressions and truth tables. However, these operations are executed using the Arduino code in which output devices, such as LED, will be activated depending on conditions and timing functions. Using digital logic design as opposed to using hardware-based systems ensures that extra ICs will not be required in the system. Thus, simplifying the process in case of modifications and tests. Generally,

this project proves the effectiveness of working together in designing projects. This is done in digital logic design and embedded systems programming using the use of Arduino.

OBJECTIVES

General Objective

The objective of this project is to create and execute a timed traffic light system using an Arduino microcontroller based on digital logic design techniques, showing a sequential control system that will not require the use of any external integrated circuit.

Specific Objectives

- Create a traffic light control system with a state-based timer sequence
- Utilize digital logic design principles such as the use of states in the development of the system behavior
- Execute the traffic lights through programming in the Arduino microcontroller and using its own timers
- Remove the need for external integrated circuits through full utilization of the microcontroller's controls
- Execute simulation of the traffic light system using LEDs as representations of the red, yellow, and green states
- Ensure system accuracy by checking the proper sequence and time execution

SYSTEM OVERVIEW

Type of System:

Sequential System (State-based timed control system)

Platform:

Arduino Uno

Inputs:

- Internal timing control (millis/delay-based sequencing)
- No external physical input devices are used in the current simulation

Outputs:

- Red LED (stop signal)
- Yellow LED (transition signal)
- Green LED (go signal)
- 7-segment display (used for countdown timing display)

Function Description:

The traffic signal system is an embedded time-based traffic controller using an Arduino microcontroller. This system follows a sequential state system whereby three states are used; red, yellow, and green. The three states are set to run for a specific amount of time depending on the timing program.

The red state means stop while the yellow means transition/care and the green indicates go. A 7-segment LCD display is used in displaying the countdown timer of each state to show the time left before the transition to another state. Overall, this is a software-controlled embedded system which does not rely on any external circuitry, making it a very flexible digital logic based system.

DIGITAL LOGIC DESIGN ANALYSIS

4.1 Problem Definition

The system tries to solve the inefficiency and inflexibility associated with traffic light control that is common with traditional traffic lights controlled by a fixed-timing system. Traditional systems have not been able to incorporate a programmable structure into their sequence logic.

This project satisfies the need for a simple traffic light system based on a microcontroller which exhibits sequential behavior derived from digital logic concepts. The system replaces complicated hardware implementations, such as integrated circuit timers and logic gates, with an efficient program implemented using the Arduino microcontroller.

4.2 Truth Table

The system operates as a sequential state machine with three main states:

STATE	Red LED (Pin 13)	Yellow LED (Pin 12)	Green LED (Pin 11)	7-Segment Display Output	Duration
S0	1	0	0	Counts 5 → 0	5 seconds
S1	0	1	0	Counts 2 → 0	2 seconds
S2	0	0	1	Counts 5 → 0	5 seconds

4.3 Boolean Expression

Let:

- S0 = Red state
- S1 = Yellow state
- S2 = Green state

Outputs:

- Red (R) = S0
- Yellow (Y) = S1
- Green (G) = S2

4.4 Simplification (Boolean Algebra / K-Map)

Since each output is active in only one state, the expressions are already in simplest form.
No further minimization is required.

4.5 Final Simplified Expression

- $R = S0$
- $Y = S1$
- $G = S2$

4.6 Logic Diagram (Equivalent Representation)

Instead of physical logic gates, the system is represented as a sequential state machine:

- $S0 \rightarrow$ Activates Red LED
- $S1 \rightarrow$ Activates Yellow LED
- $S2 \rightarrow$ Activates Green LED

State transitions occur based on timing control (Arduino delay function), forming a cyclic sequence:

$S0 \rightarrow S1 \rightarrow S2 \rightarrow S0$ (repeat)

4.7 Logic-to-Arduino Mapping

This section explains how the digital logic design is implemented in the Arduino program. Instead of using physical logic gates and ICs, the system uses software functions, loops, and digital I/O pins to represent logic behavior and state transitions.

Logic Element	Arduino Equivalent
State S0	countdown(5, redled)
State S1	countdown(2, yellowled)
State S2	countdown(5, greenled)
State Transition (S0 → S1 → S2)	Sequence in loop() function
Output R	digitalWrite(redled, HIGH)
Output Y	digitalWrite(yellowled, HIGH)
Output G	digitalWrite(greenled, HIGH)
Timing Control (Clock Signal)	delay(1000) inside for loop
Countdown Logic	for (int i = seconds; i >= 0; i--)
7-Segment Display Control	displayDigit(num) function
Segment A–G Outputs	digitalWrite(pinA–pinG, digits[num][x])
System Reset State	resetLights() function
Display Clearing	clearDisplay() function

METHODOLOGY

5.1 Design Phase

- Define the problem of inefficient fixed-timing traffic signal systems
- Design a microcontroller-based solution using Arduino to implement sequential traffic light control
- Create a state-based truth table representing RED, YELLOW, and GREEN signal transitions
- Represent system behavior using sequential logic principles (finite state machine concept)
- Implement timing-based state transitions instead of hardware-based logic circuits

5.2 Hardware Setup

- Arduino Uno microcontroller
- Breadboard for circuit assembly
- LEDs (Red, Yellow, Green) for traffic signal simulation
- 7-segment display for countdown timer visualization
- Resistors for LED and display current limiting
- Jumper wires for circuit connections
- USB cable for Arduino power and programming

Pin Configuration

7-Segment Display:

- pinA $\rightarrow$ 2

- pinB → 3
- pinC → 4
- pinD → 5
- pinE → 6
- pinF → 7
- pinG → 8

Traffic Lights:

- Green LED → Pin 11
- Yellow LED → Pin 12
- Red LED → Pin 13

5.3 Software Development

- Arduino IDE was used as the programming environment
- Program was written in C/C++ using Arduino syntax
- Code structure includes functions for:
 - LED control (resetLights())
 - 7-segment display control (displayDigit())
 - Countdown timing (countdown())
 - Logic is implemented using sequential execution inside the loop() function
 - Each traffic light state is represented by a function call with a specific time

5.4 Uploading Process

- The code is compiled using the Arduino IDE
- Arduino Uno is connected via USB cable
- Program is uploaded to the microcontroller board
- Upload success is verified through IDE confirmation message (“Done uploading”)
- System automatically starts execution after upload

5.5 Testing and Debugging

- Individual LED outputs were tested to ensure correct pin functionality
- 7-segment display was tested using number sequence validation
- Countdown timing was checked for correct 1-second intervals
- State transitions (Red → Yellow → Green) were verified in loop execution
- Errors such as incorrect wiring or display mapping were corrected during testing

MATERIALS AND TOOLS

- Arduino Uno microcontroller
- Breadboard
- Jumper wires
- Red, Yellow, Green LEDs
- 7-segment display
- Resistors (for LED and display protection)
- USB cable
- Arduino IDE software

ARDUINO SOURCE CODE

```
// SEGMENT PINS
```

```
int pinA = 2;
```

```
int pinB = 3;
```

```
int pinC = 4;
```

```
int pinD = 5;
```

```
int pinE = 6;
```

```
int pinF = 7;
```

```
int pinG = 8;
```

```
// TRAFFIC LIGHT PINS
```

```
int greenled = 11;
```

```
int yellowled = 12;
```

```
int redled = 13;
```

```
// DIGIT MAP (COMMON ANODE)
```

```
int digits[10][7] = {
```

```
{0,0,0,0,0,0,1}, // 0
```

{1,0,0,1,1,1,1}, // 1

{0,0,1,0,0,1,0}, // 2

{0,0,0,0,1,1,0}, // 3

{1,0,0,1,1,0,0}, // 4

{0,1,0,0,1,0,0}, // 5

{0,1,0,0,0,0,0}, // 6

{0,0,0,1,1,1,1}, // 7

{0,0,0,0,0,0,0}, // 8

{0,0,0,0,1,0,0} // 9 };

// SETUP

void setup() {

pinMode(pinA, OUTPUT);

pinMode(pinB, OUTPUT);

pinMode(pinC, OUTPUT);

pinMode(pinD, OUTPUT);

pinMode(pinE, OUTPUT);

```
pinMode(pinF, OUTPUT);
```

```
pinMode(pinG, OUTPUT);
```

```
pinMode(redled, OUTPUT);
```

```
pinMode(yellowled, OUTPUT);
```

```
pinMode(greenled, OUTPUT);
```

```
clearDisplay();
```

```
resetLights();
```

```
}
```

```
// RESET LIGHTS
```

```
void resetLights() {
```

```
    digitalWrite(redled, LOW);
```

```
    digitalWrite(yellowled, LOW);
```

```
    digitalWrite(greenled, LOW);
```

```
}
```

```
// CLEAR DISPLAY
```

```
void clearDisplay() {
```

```
    digitalWrite(pinA, HIGH);
```

```
digitalWrite(pinB, HIGH);
```

```
digitalWrite(pinC, HIGH);
```

```
digitalWrite(pinD, HIGH);
```

```
digitalWrite(pinE, HIGH);
```

```
digitalWrite(pinF, HIGH);
```

```
digitalWrite(pinG, HIGH);
```

```
}
```

```
// DISPLAY NUMBER
```

```
void displayDigit(int num) {
```

```
    digitalWrite(pinA, digits[num][0]);
```

```
    digitalWrite(pinB, digits[num][1]);
```

```
    digitalWrite(pinC, digits[num][2]);
```

```
    digitalWrite(pinD, digits[num][3]);
```

```
    digitalWrite(pinE, digits[num][4]);
```

```
    digitalWrite(pinF, digits[num][5]);
```

```
    digitalWrite(pinG, digits[num][6]);
```

```
}
```

```
// COUNTDOWN FUNCTION
```

```
void countdown(int seconds, int ledPin) {
```

```
    resetLights();
```

```
    digitalWrite(ledPin, HIGH);
```

```
    for (int i = seconds; i >= 0; i--) {
```

```
        displayDigit(i); delay(1000);
```

```
    }
```

```
    clearDisplay();
```

```
}
```

```
// MAIN LOOP
```

```
void loop() {
```

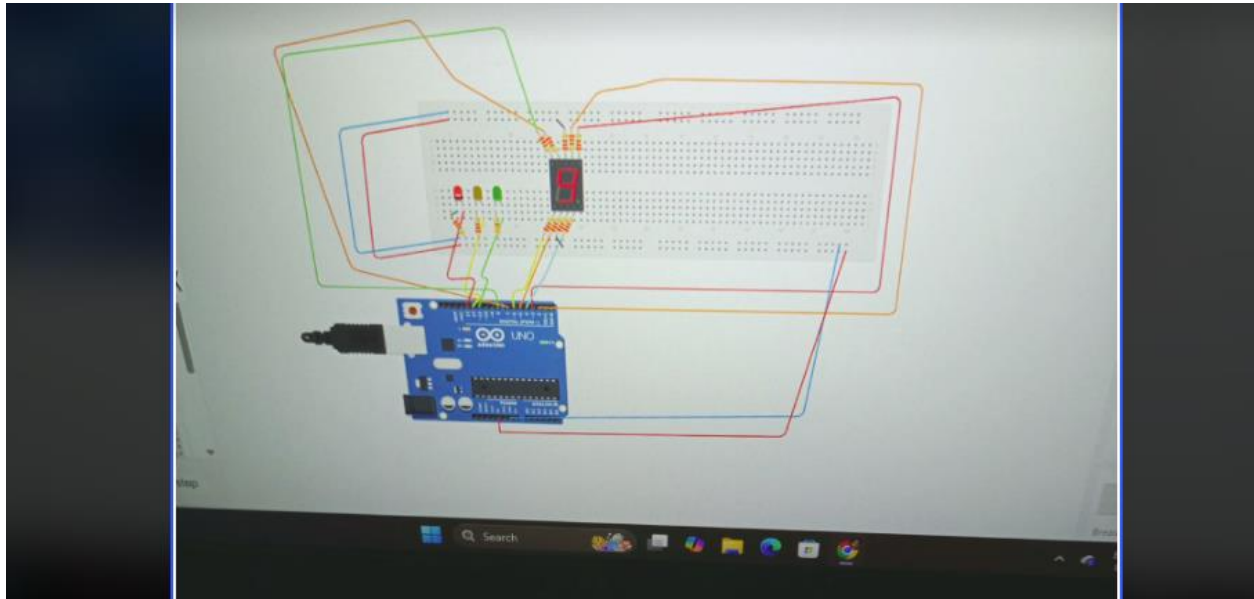
```
    countdown(5, redled);
```

```
    countdown(2, yellowled);
```

```
    countdown(5, greenled);
```

```
}
```


ARDUINO LOGIC DESIGN



CIRCUIT DESIGN / HARDWARE IMPLEMENTATION

Breadboard Diagram

In the design of this circuit, the board used is the breadboard where an Arduino Uno is the main controlling unit. This circuit contains three LED traffic lights that include the red, yellow,

and green light, along with a seven segment display used for the count down display. The connection of these components takes place using jumpers through Arduino digital pins.

Wiring Explanation

The system has two parts: the traffic light indicators and the 7-segment display.

For the traffic lights we have three LEDs. Each one is connected to a pin on the Arduino board. The red LED means stop the yellow LED means caution and the green LED means go. We put a resistor with each LED so it does not get much electricity and break.

The 7-segment display is made up of seven parts from A to G. Each part is controlled by the Arduino board. This display is special because it uses a signal to turn on the parts. The Arduino decides which parts to turn on based on the number it is showing during the countdown.

The whole system works because of the way the Arduino is programmed. It controls the traffic lights and the countdown display at the time making sure everything happens in the right order.

The traffic light. The 7-segment display work together to show us when to stop and when to go and they do this by following the instructions, from the Arduino board.

Pin Connections

7-Segment Display Connections

- Segment A → Pin 2
- Segment B → Pin 3

- Segment C → Pin 4
- Segment D → Pin 5
- Segment E → Pin 6
- Segment F → Pin 7
- Segment G → Pin 8

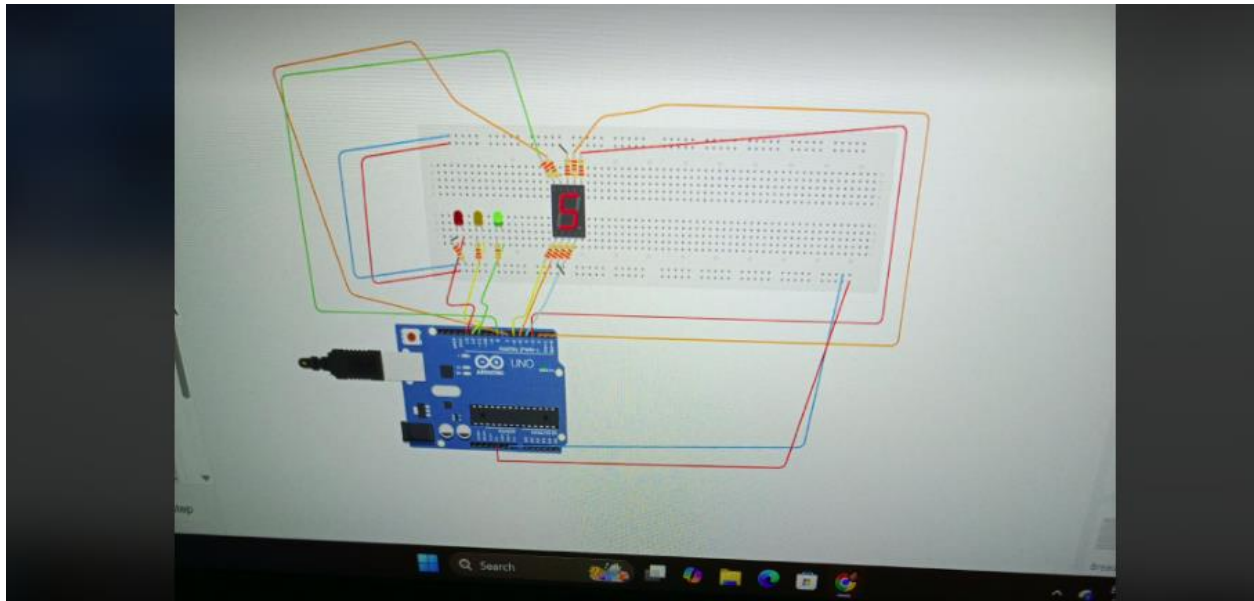
Traffic Light Connections

- Red LED → Pin 13
- Yellow LED → Pin 12
- Green LED → Pin 11

SIMULATION / ACTUAL OUTPUT

Simulation / Platform Used

The system was tested using an Arduino-based simulation environment (Tinkercad / Proteus) and/or actual hardware setup with Arduino Uno. The simulation replicates the real-world behavior of the traffic signal system, including LED sequencing and 7-segment countdown display.



Output Behavior Explanation

When the system starts, the Arduino executes a continuous loop that controls the traffic signal states in a fixed sequence:

Red Light Phase (S0)

- Red LED turns ON
- Yellow and Green LEDs remain OFF
- 7-segment display counts down from **5 to 0**
- Represents the **STOP condition** for vehicles
- Duration: 5 seconds

Yellow Light Phase (S1)

- Yellow LED turns ON
- Red and Green LEDs remain OFF
- 7-segment display counts down from **2 to 0**
- Represents the **transition / caution** phase
- Duration: 2 seconds

Green Light Phase (S2)

- Green LED turns ON
- Red and Yellow LEDs remain OFF
- 7-segment display counts down from **5 to 0**
- Represents the **GO condition** for vehicles
- Duration: 5 seconds

RESULTS AND DISCUSSION

Did Arduino Match Truth Table Outputs?

The Arduino thing worked like it was supposed to. It did what the truth table said it would do. The program had states like Red and Yellow and Green. When it was in one of these states it turned on the light and kept the others off. It went from Red to Yellow to Green just like it was supposed to. The little display that shows numbers also worked right it showed the countdown, for each state. This means that what the Arduino did is what we planned for it to do. The Arduino and the 7-segment display and the lights all worked together like they should. The Arduino implementation and the 7-segment display and the lights did what they were supposed to do.

Were There Discrepancies?

Some small problems were seen when we tested the system with how precise the timing was. The system uses a function called delay to control the timing so sometimes the timing can be a little off depending on the computer or the hardware we use.. These small problems did not change the way the system moved from one state to another or what it showed as output.

We did not find any mistakes in the way the lights on the LED were turned on and off or what was shown on the display. Everything that was supposed to happen did happen, just like we planned it to when we wrote the program, for the system.

Analysis of System Behavior

The system works in a step-by-step way using Arduino programming. It does not use computer chips instead it uses the software to control when things happen, based on timing.

Each state of the traffic light does a job in the program, so only one light is on, at a time. The countdown timer helps us see how long each state lasts and makes it clearer how the system works.

The traffic light system shows that digital logic principles can be used in a real-world setting. It acts like a state machine, which is a system that goes through a series of states in a controlled and predictable way, over and over. The traffic light system does this by cycling through its predefined states like the traffic light states in a controlled and predictable way.

PROBLEMS ENCOUNTERED

Wiring Issues

During the hardware setup we had some trouble with the 7-segment display and LEDs on the breadboard. The problem was that some pins were in the place and the jumper wires were loose. This meant that some parts of the display and some LEDs did not light up right. We solved these problems by checking the Arduino code for the pin setup again and making sure all the physical connections were secure. The 7-segment display and LEDs were connected correctly after that. The Arduino code and the physical setup matched, and everything worked properly.

Code Bugs

Several mistakes were found while developing the system. These mistakes included indexing in the 7-segment array and incorrect function calls in the display part. The outputs were not as expected at first because of these errors. Each function was tested separately to fix the issues. The system was checked until it showed the LED sequence and display output. The mistakes were

fixed by debugging each part of the system. The 7-segment array and display routine were corrected. Now the system works as expected. The LED sequence and display output are correct.

Timing Delays

The system uses a `delay()` function to control timing. This caused timing issues during simulation and testing.

- The sequence of events was still correct.
- The system is not fully optimized for real-time use because the delay function blocks other actions.

This limitation was okay because of the design approach we chose. The delay function affects how the system runs. It is not perfect for real-time systems. We knew this when we designed it.

Sensor Noise (Not Applicable)

The project did not have any issues with sensor noise. This is because the traffic signal system does not use any sensors.

The traffic signal system works with timing sequences that were set beforehand. It does not use any devices, from outside to get information. The traffic signal system is very simple. It only follows the predefined timing sequences.

CONCLUSION

Summary of Project Outcome

The project worked out well. It made a traffic signal system that has a timer. This system uses an Arduino microcontroller. The Arduino microcontroller makes the system work like a traffic light. It has yellow and green lights just like a real one. It also has a countdown display that shows numbers. This display is like the ones you see on clocks. It counts down the time, for each part of the traffic light sequence.

The people who made this project did something. They used software to control the system of using a lot of hardware. This makes the system simpler and easier to understand. The traffic signal system still works correctly with the simpler design. The Arduino microcontroller and the software make it all work together.

Success of Logic Implementation in Arduino

The Arduino project worked well. It followed the planned steps in the order based on set time intervals. The LED outputs and countdown display worked as they should have. The system showed that complex traffic control can be done using software logic. The Arduino system was reliable and efficient for logic in embedded systems. It handled the logic flow. The project was successful. The Arduino-based system proved to be good for digital logic. It did not need circuits. The system worked accurately. Overall it was a platform.

RECOMMENDATIONS

Improve Circuit Efficiency

To make the system work better we should use a way to handle time. By using the delay() function we can use millis (). This way the Arduino can do things at the same time and it will be more responsive. The Arduino will be able to perform tasks and this will make the system more efficient. Using millis() is an idea because it does not block other tasks like the delay() function does.

Add More Inputs/Outputs

The system can be made better by adding things like push buttons or infrared sensors. This will make it feel like traffic. We can use these things to control the traffic flow based on how many people're actually using the road. This is better than having a set timing for everything. We can also add things like pedestrian signals or warning buzzers to make the system feel more real. The traffic system will be more realistic with these additions, like the pedestrian signals and the warning buzzers and the external input devices such, as push buttons or infrared sensors.

Upgrade to IoT Integration

The traffic system can be improved for development by using Internet of Things technology to make a traffic control system. We can add Wi-Fi modules like ESP8266 or ESP32 to the traffic lights. This way the traffic signal can be controlled from a distance.

The traffic system can also collect information. Manage traffic in a smart way based on the Internet or cloud-based control.

The Internet of Things technology in the traffic system will help the traffic system to make changes, on its own to deal with traffic problems. The traffic system will be able to make these changes to handle traffic issues in a way.

REFERENCES

https://projecthub.arduino.cc/krishna_agarwal/traffic-light-using-arduino-a-beginner-project-35f8c6

<https://www.electronicisfun08.in/2022/12/how-to-make-traffic-lights-using-7.html>

<https://www.electronicisfun08.in/2023/05/arduino-traffic-light-autodesk.html>

https://www.youtube.com/watch?v=L-6zf2_e20U

<https://youtu.be/qR-j6sLfa1U?si=GRmNj5f3SW2LjjqB>

<https://share.google/w3VDzqP4Q7YrzeyvE>

[https://drive.google.com/drive/folders/1QL7CkHgKMS0ZxywoFg0CsXbeRkraicoE?fbclid=IwY2xjawRvmI5leHRuA2FlbQIxMABicmlkETFKUU14VDZIYU5Sa3BzU2lWc3J0YwZhcHBfaWQQMjlyMDM5MTc4ODIwMDg5MgABHsqlzmQlTkPIrOd5-](https://drive.google.com/drive/folders/1QL7CkHgKMS0ZxywoFg0CsXbeRkraicoE?fbclid=IwY2xjawRvmI5leHRuA2FlbQIxMABicmlkETFKUU14VDZIYU5Sa3BzU2lWc3J0YwZhcHBfaWQQMjlyMDM5MTc4ODIwMDg5MgABHsqlzmQlTkPIrOd5-S2SBNmaNP85lb8wvUv4y3jaBzbuYtlGqqvapCXVgLnI_aem_Sn1EuxB2KRT9YT3ZM_tvCQ)

[2xjawRvmI5leHRuA2FlbQIxMABicmlkETFKUU14VDZIYU5Sa3BzU2lWc3J0YwZhcHBfaW](https://drive.google.com/drive/folders/1QL7CkHgKMS0ZxywoFg0CsXbeRkraicoE?fbclid=IwY2xjawRvmI5leHRuA2FlbQIxMABicmlkETFKUU14VDZIYU5Sa3BzU2lWc3J0YwZhcHBfaWQQMjlyMDM5MTc4ODIwMDg5MgABHsqlzmQlTkPIrOd5-S2SBNmaNP85lb8wvUv4y3jaBzbuYtlGqqvapCXVgLnI_aem_Sn1EuxB2KRT9YT3ZM_tvCQ)

[QQMjlyMDM5MTc4ODIwMDg5MgABHsqlzmQlTkPIrOd5-](https://drive.google.com/drive/folders/1QL7CkHgKMS0ZxywoFg0CsXbeRkraicoE?fbclid=IwY2xjawRvmI5leHRuA2FlbQIxMABicmlkETFKUU14VDZIYU5Sa3BzU2lWc3J0YwZhcHBfaWQQMjlyMDM5MTc4ODIwMDg5MgABHsqlzmQlTkPIrOd5-S2SBNmaNP85lb8wvUv4y3jaBzbuYtlGqqvapCXVgLnI_aem_Sn1EuxB2KRT9YT3ZM_tvCQ)

[S2SBNmaNP85lb8wvUv4y3jaBzbuYtlGqqvapCXVgLnI_aem_Sn1EuxB2KRT9YT3ZM_tvCQ](https://drive.google.com/drive/folders/1QL7CkHgKMS0ZxywoFg0CsXbeRkraicoE?fbclid=IwY2xjawRvmI5leHRuA2FlbQIxMABicmlkETFKUU14VDZIYU5Sa3BzU2lWc3J0YwZhcHBfaWQQMjlyMDM5MTc4ODIwMDg5MgABHsqlzmQlTkPIrOd5-S2SBNmaNP85lb8wvUv4y3jaBzbuYtlGqqvapCXVgLnI_aem_Sn1EuxB2KRT9YT3ZM_tvCQ)

https://drive.google.com/drive/folders/1wPXsQ8k3rOxx-Rev4ArxGbRNwLJbVyvw?fbclid=IwY2xjawRvnFBleHRuA2FlbQIxMABicmlkETFKUU14VDZIYU5Sa3BzU2lWc3J0YwZhcHBfaWQQMjIyMDM5MTc4ODIwMDg5MgABHucp6w4TMoJyPiwbgnG2lGg7kA1uDwqtM3SMUHOS0hhF6SgGiXDobwAy_YNm_aem_HsZ0NHZlunJsoV3e8us8OA

https://drive.google.com/drive/u/0/folders/1ykHpuSP5foqDVpJVhO3V_cBt3gJ_0hu4?fbclid=IwY2xjawRvnIzleHRuA2FlbQIxMABicmlkETFKUU14VDZIYU5Sa3BzU2lWc3J0YwZhcHBfaWQQMjIyMDM5MTc4ODIwMDg5MgABHmIbNyDrTlMyabgu9teKbBMGBbBrdxmamf8t5wk88a1h_tyadF_flIz6pQiy_aem_1RnwriuS5CkluBSA6VzNWw

https://drive.google.com/drive/u/1/folders/1eJ8R6UmFyh7hO6nM21JXZzi6R1f_XVc-

APPENDICES

Appendix A – Arduino Source Code

```
// SEGMENT PINS
```

```
int pinA = 2;
```

```
int pinB = 3;
```

```
int pinC = 4;
```

```
int pinD = 5;
```

```
int pinE = 6;
```

```
int pinF = 7;
```

```
int pinG = 8;
```

```
// TRAFFIC LIGHT PINS
```

```
int greenled = 11;
```

```
int yellowled = 12;
```

```
int redled = 13;
```

```
// DIGIT MAP (COMMON ANODE)
```

```
int digits[10][7] = {
```

```
{0,0,0,0,0,0,1}, // 0
```

```
{1,0,0,1,1,1,1}, // 1
```

```
{0,0,1,0,0,1,0}, // 2
```

```
{0,0,0,0,1,1,0}, // 3
```

```
{1,0,0,1,1,0,0}, // 4
```

```
{0,1,0,0,1,0,0}, // 5
```

```
{0,1,0,0,0,0,0}, // 6
```

```
{0,0,0,1,1,1,1}, // 7
```

```
{0,0,0,0,0,0,0}, // 8
```

```
{0,0,0,0,1,0,0} // 9 };
```

```
// SETUP
```

```
void setup() {
```

```
  pinMode(pinA, OUTPUT);
```

```
  pinMode(pinB, OUTPUT);
```

```
  pinMode(pinC, OUTPUT);
```

```
  pinMode(pinD, OUTPUT);
```

```
  pinMode(pinE, OUTPUT);
```

```
  pinMode(pinF, OUTPUT);
```

```
  pinMode(pinG, OUTPUT);
```

```
  pinMode(redled, OUTPUT);
```

```
  pinMode(yellowled, OUTPUT);
```

```
  pinMode(greenled, OUTPUT);
```

```
  clearDisplay();
```

```
  resetLights();
```

```
}
```

```
// RESET LIGHTS
```

```
void resetLights() {  
  
    digitalWrite(redled, LOW);  
  
    digitalWrite(yellowled, LOW);  
  
    digitalWrite(greenled, LOW);  
  
}
```

```
// CLEAR DISPLAY
```

```
void clearDisplay() {  
  
    digitalWrite(pinA, HIGH);  
  
    digitalWrite(pinB, HIGH);  
  
    digitalWrite(pinC, HIGH);  
  
    digitalWrite(pinD, HIGH);  
  
    digitalWrite(pinE, HIGH);  
  
    digitalWrite(pinF, HIGH);  
  
    digitalWrite(pinG, HIGH);  
  
}
```

```
// DISPLAY NUMBER
```

```
void displayDigit(int num) {
```



```
digitalWrite(pinA, digits[num][0]);

digitalWrite(pinB, digits[num][1]);

digitalWrite(pinC, digits[num][2]);

digitalWrite(pinD, digits[num][3]);

digitalWrite(pinE, digits[num][4]);

digitalWrite(pinF, digits[num][5]);

digitalWrite(pinG, digits[num][6]);

}
```

```
// COUNTDOWN FUNCTION
```

```
void countdown(int seconds, int ledPin) {

resetLights();

digitalWrite(ledPin, HIGH);

for (int i = seconds; i >= 0; i--) {

displayDigit(i); delay(1000);

}

clearDisplay();
```

```
}
```

```
// MAIN LOOP
```

```
void loop() {
```

```
  countdown(5, redled);
```

```
  countdown(2, yellowled);
```

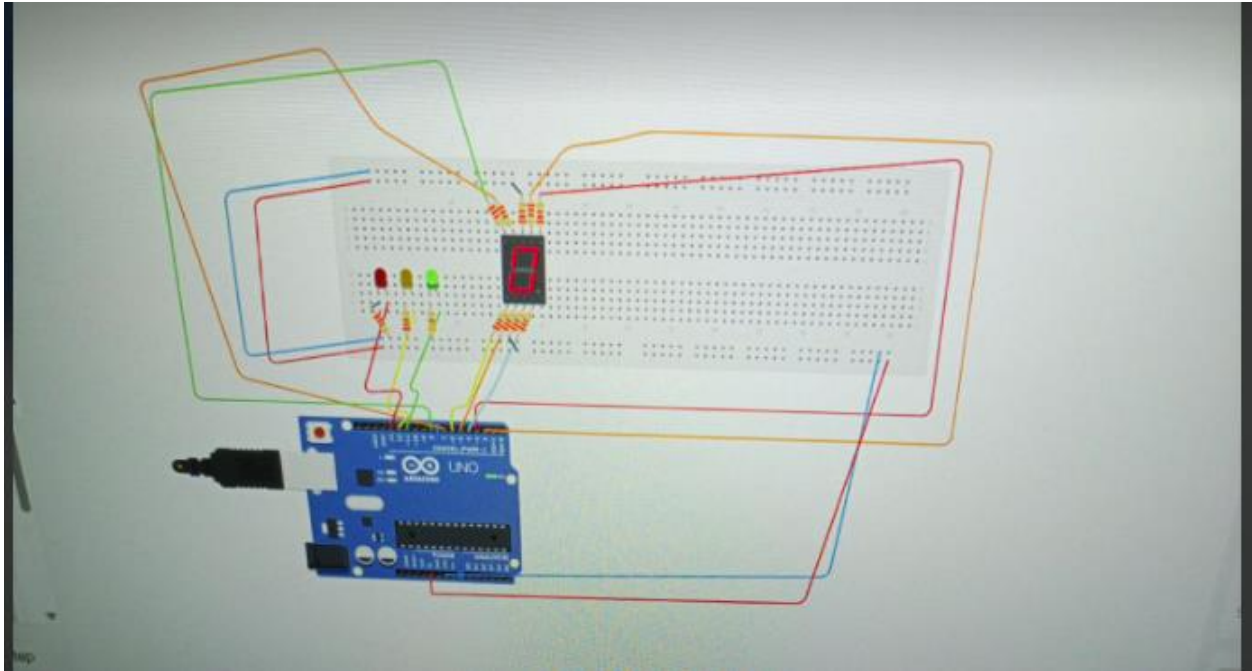
```
  countdown(5, greenled);
```

```
}
```

Appendix B – Circuit Diagram

- Arduino UNO
- LED
- 220 Ω Resistor
- Breadboard
- Jumper Wires

Note: Add a image below!



Appendix C – Components and Materials

Component	Quantity	Description
Arduino UNO	1	Main microcontroller used to control the system
Breadboard	1	Used for temporary circuit connections
LED	3	Serves as output indicator lights
220 Ω Resistor	10	Prevents excessive current flow to LEDs
Jumper Wires	Several	Connects components in the circuit
USB Cable	1	Used to upload code and

		power the Arduino
--	--	-------------------

Appendix D – Truth Table

STATE	Red LED (Pin 13)	Yellow LED (Pin 12)	Green LED (Pin 11)	7-Segment Display Output	Duration
S0	1	0	0	Counts 5 → 0	5 seconds
S1	0	1	0	Counts 2 → 0	2 seconds
S2	0	0	1	Counts 5 → 0	5 seconds